

数据库工程作业

要求:

- 1. 完成一个小型的数据库信息管理系统（或部分功能），并填写工程作业报告；程序和报告请在规定时间之内上传。
- 2. 开发模式（B/S 或 C/S）、开发高级语言任选，后台数据库使用大型数据库管理系统（SQL Server、Oracle、MySQL 等），不要使用桌面数据库。
- 3. 报告中所列举的四种操作，每种操作举一个例子即可。
- 4. 作业成绩按照报告中的标准评分，程序只实现报告中涉及的部分即可。
- 5. 作业完成后，请将工程作业报告和程序打包提交给助教老师，并联系助教老师进行系统说明和演示，回答相关问题。

工程作业报告

1. 项目信息（10 分）

学号	2312331	姓名	王一诺	专业	计算机科学与技术
项目名称	哔哩哔哩番剧管理系统				
必备环境	MySQL 、Python 、Flask、Bootstrap 5、html/css/js				
系统主要功能简介（4 分）	1. 番剧信息管理（增删改查、标签关联、播放量统计、多指标查询） 2. 番剧与制作单位、导演、声优、原作信息管理 3. 多对多关系维护（制作、导演、声优参演、改编关系） 4. 排行榜与统计（播放量/评分 Top10、制作单位番剧数量统计） 5. 数据审计（操作日志、变更历史）				
系统主要页面截图（6 分）	Bilibili番剧库 首页 新增番剧 制作单位 统计 登出 Please log in to access this page. 登录 用户名 admin 密码 登录				

番剧列表

按名称搜索

地区

全部标签

评分

☒ 降序

筛选

ID	名称	播放量	评分	创建时间	操作
4	孤独摇滚!	200000000	9.9	2025-05-25 21:41:01	编辑 删除
5	轻音少女 第一季	937030000	9.9	2025-05-25 21:46:02	编辑 删除
8	凸变英雄X	95748000	9.6	2025-05-25 21:56:31	编辑 删除
6	关于我转生变成史莱姆这档事	510000000	9.4	2025-05-25 21:49:09	编辑 删除
7	凹凸世界 第一季	190000000	8.8	2025-05-25 21:53:38	编辑 删除

孤独摇滚!

年份: 2022 地区: 日本

播放量: 200000000 评分: 9.9

快捷操作

[编辑番剧](#)

[删除番剧](#)

[返回列表](#)

标签管理

[改编](#) [日常](#) [音乐](#)

新增标签

添加

导演管理

[高藤圭一郎](#)

高藤圭一郎

添加导演

声优管理

[青山吉能](#) [铃代纱弓](#) [张碧娟](#)

青山吉能

添加声优

改编管理

漫画改 — 《孤独摇滚!》 by 滨路晶

Keion!

kakifly

改编类型

添加

Bilibili番剧库

首页 新增番剧 制作单位 统计 登出

新增番剧

名称 *

年份 *

地区 *

播放量 *

0

评分

声优 1

输入声优名称

声优 2

输入声优名称

声优 3

输入声优名称

标签 1

一无一

标签 2

一无一

标签 3

一无一

导演

输入导演名称

制作单位

输入制作单位名称

是否改编动画

否

☒ 是

原作名称 *

输入原作名称

原作者 *

输入原作者姓名

地区

可选

改编类型

漫画改/小说改/游戏改...

Bilibili番剧库

首页 新增番剧 制作单位 统计 登出

播放量 Top10

排名	番剧	播放量
1	轻音少女 第一季	937030000
2	关于我转生变成史莱姆这档事	510000000
3	孤独摇滚!	200000000
4	凹凸世界 第一季	190000000
5	凸变英雄X	95748000

Bilibili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

操作审计日志

ID	表名	操作	记录ID	旧数据	新数据
246	tag	UPDATE	小说改	{"usage_count": 0}	{"usage_cour
247	anime_tag	INSERT	6 小说改		{"anime_id":
244	tag	UPDATE	小说改	{"usage_count": 1}	{"usage_cour
245	anime_tag	DELETE	6 小说改	{"anime_id": 6, "tag_name": "小说改"}	
241	tag	INSERT	小说改		{"usage_cour
242	tag	UPDATE	小说改	{"usage_count": 0}	{"usage_cour
243	anime_tag	INSERT	6 小说改		{"anime_id":
239	tag	UPDATE	穿越	{"usage_count": 0}	{"usage_cour
240	anime_tag	INSERT	6 穿越		{"anime_id":

2. 系统配置（10 分）

说明		(2 分) 请说明系统配置情况（后台数据库，高级语言）； (8 分) 请使用连接串连接高级语言和数据库，并分析字符串的各个部分。		
配置 步骤 2 分	DBMS	1. MySQL		
		2.		
	高级 语言	1. Python		
		2. SQL		
连接串 分析 (6 分)	序号	名称	功能说明	取值
	1	db	填写在 config.py 文件中，用于与 MySQL 连接，接上我们的数据库 bilibili_anime	无
连接串代码 (截屏) (2 分)	<pre>MYSQL_HOST = 'localhost' MYSQL_USER = 'root' MYSQL_PASSWORD = ' ' MYSQL_DB = 'bilibili_anime' MYSQL_CURSORCLASS = 'DictCursor' SECRET_KEY = ' ' </pre>			
备注	连接过程用到了数据库的账号密码。此外还要注意数据库的名称不要输错。			

3. 数据库设计（14 分）

说明	(10 分) 按照数据表的创建顺序，依次给出所涉及数据表的信息，其中参照字段以“（字段 1，字段 2，……，字段 n）”的形式给出，被参照字段以“表名（字段 1，字段 2，……，字段 n）”的形式给出； (4 分) 一般 DBMS 都可以为数据库生成关系图，请将该图片截屏并粘贴到表格中。
----	--

数据表 (10)	创建顺序	数据表名称	主键	参照属性	被参照表及属性
	1	anime	anime_id	无	无
	2	studio	name	无	无
	3	tag	name	无	无
	4	director	director_id	无	无
	5	seiyuu	seiyuu_id	无	无
	6	original_work	(title, author)	无	无
	7	adapted_anime	anime_id	original_title, original_author	original_work(title, author)
	8	production	(anime_id, studio_name)	anime_id	anime(anime_id)
	9	directorship	(director_id, anime_id)	director_id	director(director_id)
	10	anime_tag	(tag_name, anime_id)	(tag_name) (anime_id)	tag(name) anime(anime_id)
	11	casting	(seiyuu_id, anime_id)	(seiyuu_id) (anime_id)	seiyuu(seiyuu_id) anime(anime_id)
关系图 (4)					
备注					

4. 含有事务应用的删除操作（13分）

说明	(1分) 简要说明该操作所要完成的功能； (2分) 该操作会涉及的表（必须含有两张或两张以上的关系表，同时以“表名”的形式给出） (1分) 表连接涉及字段描述（描述方式为“表1.属性=表2.属性”） (1分) 删除条件涉及的字段描述（以“表名.属性=?”形式给出）
----	--

	(4分) 实现该操作的关键代码（高级语言、SQL），截图即可；（其中如果删除语句中不包含任何形式的事务应用将扣除3分） (4分) 如何执行该操作，按所述方法能够正常演示程序则给分。																																					
功能描述 (1分)	实现番剧数据的完整删除功能，确保删除主表记录时同步删除所有关联的子表（改编动画信息）和关系表（标签、制作单位、声优等关联数据），保证数据一致性和事务原子性。																																					
涉及的表 (2分)	anime（番剧主表）、adapted_anime（改编动画子表） 关系表：production（制作单位关联表）、anime_tag（标签关联表）、casting（声优参演表）、directorship（导演关联表） operation_audit（审计表）																																					
表连接涉及字段 (1分)	anime.anime_id 与 adapted_anime.anime_id 、 production.anime_id 、 anime_tag.anime_id、casting.anime_id、directorship.anime_id																																					
删除条件 字段描述 (1分)	字段	规则																																				
	anime.anime_id =[目标番剧 ID]	主键匹配，确保精准定位待删除记录。																																				
代码 (4分)	<pre>def delete_anime(anime_id): """ 删除番剧 """ cur = db.connection.cursor() cur.execute('DELETE FROM anime WHERE anime_id=%s', (anime_id,)) db.connection.commit() # 隐式事务提交</pre> 因为使用了MySQL 的 InnoDB 引擎，它会自动处理外键级联删除。																																					
程序演示 (4分)	Bilibili番剧库 首页 新增番剧 制作单位 统计 登出 番剧列表 按名称搜索 地区 全部标签 默认排序 ☐ 降序 筛选 <table><tr><th>ID</th><th>名称</th><th>播放量</th><th>评分</th><th>创建时间</th><th>操作</th></tr><tr><td>4</td><td>孤独摇滚!</td><td>200000000</td><td>9.9</td><td>2025-05-25 21:41:01</td><td>编辑 删除</td></tr><tr><td>5</td><td>轻音少女 第一季</td><td>937030000</td><td>9.9</td><td>2025-05-25 21:46:02</td><td>编辑 删除</td></tr><tr><td>6</td><td>关于我转生成史莱姆这档事</td><td>510000000</td><td>9.4</td><td>2025-05-25 21:49:09</td><td>编辑 删除</td></tr><tr><td>7</td><td>凹凸世界 第一季</td><td>190000000</td><td>8.8</td><td>2025-05-25 21:53:38</td><td>编辑 删除</td></tr><tr><td>8</td><td>凸变英雄X</td><td>95748000</td><td>9.6</td><td>2025-05-25 21:56:31</td><td>编辑 删除</td></tr></table> 我们点击凹凸世界第一季的删除按钮：		ID	名称	播放量	评分	创建时间	操作	4	孤独摇滚!	200000000	9.9	2025-05-25 21:41:01	编辑 删除	5	轻音少女 第一季	937030000	9.9	2025-05-25 21:46:02	编辑 删除	6	关于我转生成史莱姆这档事	510000000	9.4	2025-05-25 21:49:09	编辑 删除	7	凹凸世界 第一季	190000000	8.8	2025-05-25 21:53:38	编辑 删除	8	凸变英雄X	95748000	9.6	2025-05-25 21:56:31	编辑 删除
ID	名称	播放量	评分	创建时间	操作																																	
4	孤独摇滚!	200000000	9.9	2025-05-25 21:41:01	编辑 删除																																	
5	轻音少女 第一季	937030000	9.9	2025-05-25 21:46:02	编辑 删除																																	
6	关于我转生成史莱姆这档事	510000000	9.4	2025-05-25 21:49:09	编辑 删除																																	
7	凹凸世界 第一季	190000000	8.8	2025-05-25 21:53:38	编辑 删除																																	
8	凸变英雄X	95748000	9.6	2025-05-25 21:56:31	编辑 删除																																	

删除成功

番剧列表

按名称搜索

地区

全部标签

默认排序

☐ 降序

筛选

ID	名称	播放量	评分	创建时间	操作
4	孤独摇滚!	200000000	9.9	2025-05-25 21:41:01	编辑 删除
5	轻音少女 第一季	937030000	9.9	2025-05-25 21:46:02	编辑 删除
6	关于我转生变成史莱姆这档事	510000000	9.4	2025-05-25 21:49:09	编辑 删除
8	凸变英雄X	95748000	9.6	2025-05-25 21:56:31	编辑 删除

可以看到弹出了成功删除的提示，且列表里凹凸世界的记录消失了。

我们去它的制作单位-七创设，以及统计榜单下查看，可见这部番的记录都消失了：

← 返回制作单位列表

七创社（共 0 部）

ID	名称	播放量	评分
----	----	-----	----

播放量 Top10

排名	番剧	播放量
1	轻音少女 第一季	937030000
2	关于我转生变成史莱姆这档事	510000000
3	孤独摇滚!	200000000
4	凸变英雄X	95748000

备注

5. 触发器控制下的添加操作（20 分）

说明	（1 分）简要说明该操作所要完成的功能； （2 分）简要说明该触发器所要完成的功能 （1 分）该操作会涉及的表（以“表名”的形式给出）。 （2 分）该操作输入数据以及输入数据应该满足的条件，如：数值范围、是否为空； （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （8 分）如何执行该操作，按所述方法能够正常演示程序则给分。	
功能描述 （1 分）	为番剧添加标签时，如果欲添加标签是已有标签库里存在的则直接添加，若不存在则新建标签并添加。同时自动更新标签使用计数和操作审计日志。	
触发器描述 （2 分）	trg_tag_increment: 插入 anime_tag 时，对应标签的 usage_count +1; audit_tag_after_ins 与 audit_tag_after_upd: 新增 tag 插入 tag 表或 tag 表更新时，操作日志对应记录更新。	
涉及的表 （1 分）	anime_tag（关系表） tag（标签表） operation_audit（审计表）	
输入数据 （2 分）	字段	规则
	tag_name	存在于 tag.name 中，若不存在则新建 tag
	anime_id	必须存在于 anime.anime_id 中
插入操作 源码 （3 分）	在我们的主入口文件中定义了处理函数 add_tag 管理为单个番剧的标签添加操作： <pre> # 标签管理 @app.route('/anime/<int:anime_id>/tag/add', methods=['POST']) @login_required def add_tag(anime_id): name = request.form['tag'] models.link_tag(anime_id, name) return redirect(url_for('detail', anime_id=anime_id)) </pre> 用到了关联函数 link_tag,用于关联番剧表和标签表以及二者的关系表,定义如下： <pre> def link_tag(anime_id, tag_name): """ 为番剧关联标签 """ try: cur = db.connection.cursor() # 1. 确保标签存在（若不存在则插入） cur.execute("INSERT IGNORE INTO tag (name) VALUES (%s)", (tag_name,)) # 2. 插入关联记录，触发 trg_tag_increment cur.execute("INSERT INTO anime_tag (tag_name, anime_id) VALUES (%s, %s)", (tag_name, anime_id)) db.connection.commit() except Exception as e: db.connection.rollback() raise e </pre> 这样就完整考虑到了标签是否存在的问题。	

触发器源码
(3 分)

```
-- ===== 3. TAG 表审计 =====
CREATE TRIGGER audit_tag_after_ins
AFTER INSERT ON tag
FOR EACH ROW
BEGIN
    INSERT INTO operation_audit(table_name,operation,record_id,new_data)
    VALUES('tag','INSERT',NEW.name,
           JSON_OBJECT('usage_count',NEW.usage_count));
END$$

CREATE TRIGGER audit_tag_after_upd
AFTER UPDATE ON tag
FOR EACH ROW
BEGIN
    INSERT INTO operation_audit(table_name,operation,record_id,old_data,new_data)
    VALUES('tag','UPDATE',OLD.name,
           JSON_OBJECT('usage_count',OLD.usage_count),
           JSON_OBJECT('usage_count',NEW.usage_count));
END$$

-- 在 anime_tag 插入/删除时维护 tag.usage_count
DELIMITER $$
CREATE TRIGGER trg_tag_increment AFTER INSERT ON anime_tag
FOR EACH ROW
BEGIN
    UPDATE tag
    SET usage_count = usage_count + 1
    WHERE name = NEW.tag_name;
END$$
```

定义触发器用于对 tag 以及 anime_tag 的插入操作后的级联操作，包括对标签使用数和操作审计日志的更新。

程序演示
(4 分)



为《孤独摇滚！》添加“漫画改”标签，这是一个原有标签库里不存在的标签：

孤独摇滚！

年份：2022 地区：日本

播放量：200000000 评分：9.9

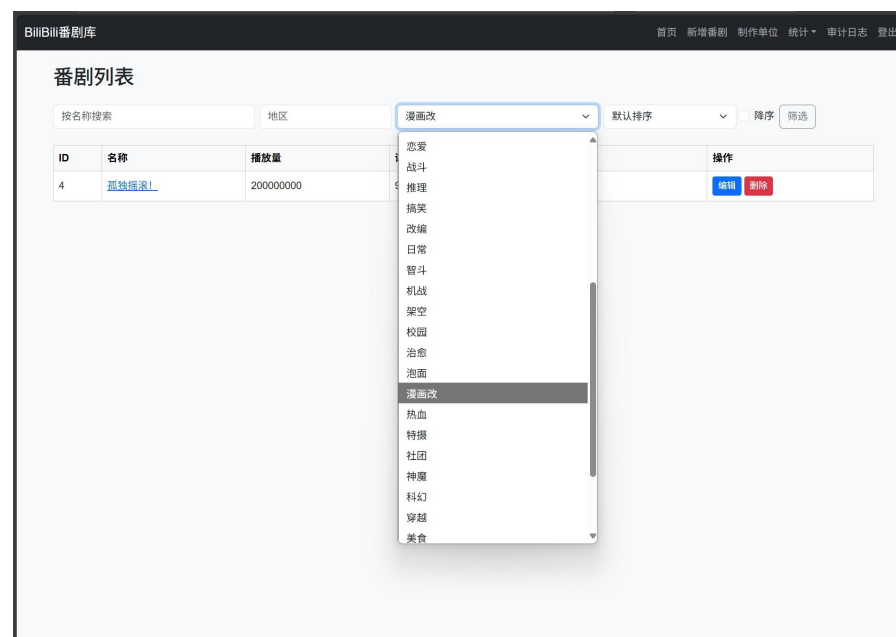
标签管理

少女 改编 日常 漫画改 音乐

新增标签

添加

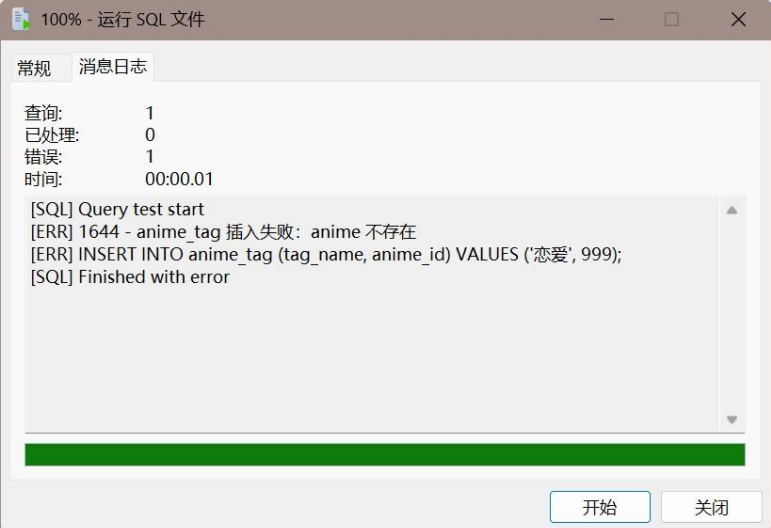
成功添加，同时我们看看现在的可选标签列表：



可见新增“漫画改”的操作也成功触发。再看看操作日志：

Bilibili番剧库						
操作审计日志						
ID	表名	操作	记录ID	旧数据	新数据	时间
248	tag	INSERT	漫画改		{"usage_count": 0}	2025-05-26 19:18:33
249	tag	UPDATE	漫画改	{"usage_count": 0}	{"usage_count": 1}	2025-05-26 19:18:33
250	anime_tag	INSERT	4 漫画改		{"anime_id": 4, "tag_name": "漫画改"}	2025-05-26 19:18:33
246	tag	UPDATE	小说改	{"usage_count": 0}	{"usage_count": 1}	2025-05-26 18:34:10
247	anime_tag	INSERT	6 小说改		{"anime_id": 6, "tag_name": "小说改"}	2025-05-26 18:34:10
244	tag	UPDATE	小说改	{"usage_count": 1}	{"usage_count": 0}	2025-05-26 18:33:25
245	anime_tag	DELETE	6 小说改	{"anime_id": 6, "tag_name": "小说改"}		2025-05-26 18:33:25
241	tag	INSERT	小说改		{"usage_count": 0}	2025-05-26 18:33:02
242	tag	UPDATE	小说改	{"usage_count": 0}	{"usage_count": 1}	2025-05-26 18:33:02
243	anime_tag	INSERT	6 小说改		{"anime_id": 6, "tag_name": "小说改"}	2025-05-26 18:33:02
239	tag	UPDATE	穿越	{"usage_count": 0}	{"usage_count": 1}	2025-05-26 18:32:54

也成功触发了日志记录更新。

程序演示 (4 分)	说明：违背触发器要求，不能够执行插入操作，系统报错。 让我们执行以下 sql 语句： <pre>test.sql 1 INSERT INTO anime_tag (tag_name, anime_id) VALUES ('恋爱', 999);</pre> 这个语句尝试为不存在的番剧（anime_id=999）添加标签“恋爱”，我们执行它：  可见插入失败，报错：“anime 不存在”
备注	

6. 存储过程控制下的更新操作（18分）

说明	（1分）简要说明该操作所要完成的功能； （1分）简要说明该存储过程所要完成的功能； （2分）说明该操作涉及操作的表（必须包含两张或两张以上的关系表，以“表名形式”描述） （1分）表连接涉及字段描述（描述方式为“表 1. 属性=表 2. 属性”） （2分）该操作会修改字段（以“表名. 字段名”的形式给出），以及修改规则，如新数值的计算方法、在何种条件下予以修改等； （6分）实现该操作的关键代码（高级语言、SQL），截图即可； （5分）如何执行该操作，按所述方法能够正常演示程序则给分。	
功能描述（1分）	新增一个专门用于“通过存储过程更新番剧评分”的功能，数据库层校验评分范围并自动写审计日志，后端路由调用存储过程，前端在详情页直接提交更新。	
存储过程功能描述（1分）	用户在番剧详情页通过表单输入新的评分值，后端将调用定义的存储过程 sp_update_rating，该过程会先验证评分是否在 0~10 的合法范围内，若合法则更新对应番剧的 rating 字段，并将此次操作以 JSON 格式写入 operation_audit 审计日志表中。	
涉及的关系表（2分）	主表：anime 审计表：operation_audit	
表连接涉及字段（1分）	anime.anime_id：主键字段，用于定位要更新的番剧记录； operation_audit.record_id：记录更新日志时，存储对应的 anime.anime_id，作为变更对象的唯一标识。	
更改字段（2分）	字段	规则
	p_anime_id	目标番剧主键必须在 anime.anime_id 中存在
	p_new_rating	新评分（0.0~10.0），需满足 $0 \leq p_new_rating \leq 10$
更新代码（3分）	<pre> #更新评分 @app.route('/anime/<int:anime_id>/rating', methods=['POST']) @login_required def update_rating(anime_id): """ 使用存储过程更新评分 """ # 参数校验 try: new_rating = float(request.form.get('new_rating')) except (TypeError, ValueError): flash('请输入合法的评分数值', 'danger') return redirect(url_for('detail', anime_id=anime_id)) try: models.call_sp_update_rating(anime_id, new_rating) flash('评分已更新(via 存储过程)', 'success') except Exception as e: flash(f'更新失败: {e}', 'danger') return redirect(url_for('detail', anime_id=anime_id)) </pre>	

创建存储过程源码（3分）

```
-- =====
-- 5. 存储过程控制下的更新操作
-- =====

DELIMITER $$
CREATE PROCEDURE sp_update_rating(
    IN p_anime_id INT,
    IN p_new_rating DECIMAL(3,1)
)
BEGIN
    -- 验证评分范围
    IF p_new_rating < 0 OR p_new_rating > 10 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT='评分必须在 0 到 10 之间';
    END IF;
    -- 执行更新
    UPDATE anime
        SET rating = p_new_rating
        WHERE anime_id = p_anime_id;
    -- 写审计日志
    INSERT INTO operation_audit(table_name, operation, record_id, new_data)
        VALUES(
            'anime','UPDATE',CAST(p_anime_id AS CHAR),
            JSON_OBJECT('rating', p_new_rating)
        );
END$$
DELIMITER ;
```

存储过程执行源码（1分）

```
def call_sp_update_rating(anime_id, new_rating):
    """
    调用存储过程更新番剧评分
    """
    cur = db.connection.cursor()
    # 调用存储过程
    cur.callproc('sp_update_rating', (anime_id, new_rating))
    db.connection.commit()
```

程序演示（2分）

说明：不违背存储过程，能够执行更新操作
在更新前，我们先记录一下现在的评分榜：

Bilibili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

评分 Top10

排名	番剧	评分
1	孤独摇滚!	9.9
2	轻音少女 第一季	9.9
3	凸变英雄X	9.6
4	关于我转生成史莱姆这档事	9.4

然后我们尝试对《孤独摇滚！》做评分更新操作：

Bilibili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

孤独摇滚！

年份：2022 地区：日本

播放量：200000000 评分：9.9

快速操作

编辑番剧

删除番剧

返回列表

评分

9.9

9.8

更新评分

标签管理

少女 改编 日常 漫画改 音乐

新增标签

添加

导演管理

高杉圭一郎

高杉圭一郎

添加导演

声优管理

青山吉能 古代玲奈 大西纯

青山吉能

添加声优

更新后我们查看现在的首页：

Bilibili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

番剧列表

按名称搜索 地区 全部标签 评分 降序 筛选

ID	名称	播放量	评分	创建时间	操作
5	轻音少女 第一季	937030000	9.9	2025-05-25 21:46:02	编辑 删除
4	孤独摇滚！	200000000	9.8	2025-05-25 21:41:01	编辑 删除
8	凸变英雄X	95748000	9.6	2025-05-25 21:56:31	编辑 删除
6	关于我转生变成史莱姆这档事	510000000	9.4	2025-05-25 21:49:09	编辑 删除

更新成功，而且按评分降序查询也正常显示了更新后的查询结果。我们再去看看评分榜：

Bilibili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

评分 Top10

排名	番剧	评分
1	轻音少女 第一季	9.9
2	孤独摇滚！	9.8
3	凸变英雄X	9.6
4	关于我转生变成史莱姆这档事	9.4

可以看到也随之更新了，《孤独摇滚！》更新到了《轻音少女第一季》后面。

程序演示
(2分)

说明：违背存储过程，系统报错；

我们尝试给《孤独摇滚！》更新成一个非法分数，比如说 11：

备注	Bilibili番剧库 首页新增番剧制作单位统计▼审计日志登出 孤独摇滚！ 年份：2022 地区：日本 播放量：200000000 评分：9.8 评分9.811更新评分 标签管理 少女 改编 日常 漫画改 音乐 新增标签添加 快捷操作 编辑番剧删除番剧返回列表
	此时会有弹窗警告，且更新操作也没有执行： Bilibili番剧库 首页新增番剧制作单位统计▼审计日志登出 孤独摇滚！ 年份：2022 地区：日本 播放量：200000000 评分：9.8 评分9.811更新评分 标签管理 少女 改编 日常 漫画改 音乐 新增标签添加 快捷操作 编辑番剧删除番剧返回列表

7. 含有视图的查询操作（15 分）

说明	（1 分）简要说明该操作所要完成的功能； （1 分）简要说明建立的该视图的功能； （2 分）简要说明该操作涉及的关系数据表（以“表名”的形式给出） （1 分）简要说明表连接涉及的字段（以“表 1. 属性=表 2. 属性”） （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （4 分）如何执行该操作，按所述方法能够正常演示程序则给分。
操作功能描述(1分)	快速读取播放量 Top10、评分 Top10、制作单位番剧统计。
视图功能描述(1分)	view_top10_by_views: 按 views 降序取前 10 条。 view_top10_by_rating: 按 rating 降序取前 10 条。 view_studio_counts: 统计每个 studio_name 对应的番剧数量。
涉及的关系表(2分)	anime、production。
表连接字段（1 分）	production.studio_name = studio.name（统计制作单位名称） view_top10_by_views/view_top10_by_rating 仅查询 anime。
创建视图代码(3分)	<pre>-- 3.1 播放量 Top 10 CREATE OR REPLACE VIEW view_top10_by_views AS SELECT anime_id, title, views FROM anime ORDER BY views DESC LIMIT 10; -- 3.2 评分 Top 10 CREATE OR REPLACE VIEW view_top10_by_rating AS SELECT anime_id, title, rating FROM anime WHERE rating IS NOT NULL ORDER BY rating DESC LIMIT 10; -- 3.3 制作单位番剧统计 CREATE OR REPLACE VIEW view_studio_counts AS SELECT p.studio_name, COUNT(*) AS anime_count FROM production p GROUP BY p.studio_name;</pre>

查询代码
(3 分)

```
# 排行榜: 播放量 Top10
@app.route('/rank/views')
@login_required
def rank_views():
    top = models.get_top10_by_views()
    return render_template('rank_views.html', top=top)

# 排行榜: 评分 Top10
@app.route('/rank/rating')
@login_required
def rank_rating():
    top = models.get_top10_by_rating()
    return render_template('rank_rating.html', top=top)

# 制作单位统计
@app.route('/stats/studios')
@login_required
def stats_studios():
    counts = models.get_studio_counts()
    return render_template('stats_studios.html', counts=counts)

# 获取播放量 Top10
def get_top10_by_views():
    cur = db.connection.cursor()
    cur.execute('SELECT * FROM view_top10_by_views')
    return cur.fetchall()

# 获取评分 Top10
def get_top10_by_rating():
    cur = db.connection.cursor()
    cur.execute('SELECT * FROM view_top10_by_rating')
    return cur.fetchall()

# 获取各制作单位番剧数量
def get_studio_counts():
    cur = db.connection.cursor()
    cur.execute('SELECT * FROM view_studio_counts')
    return cur.fetchall()
```

程序演示
(4 分)

上边栏里的“统计”下拉有“播放量 Top10”、“评分 Top10”和“制作单位统计”三个选项，依次点击可以看到以下视图：（由于现在数据库中的番剧数目没超过 10，所以其余为空）

Bilibili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

播放量 Top10

排名	番剧	播放量
1	轻音少女 第一季	937030000
2	关于我转生成史莱姆这件事	510000000
3	孤独摇滚!	200000000
4	凸变英雄X	95748000

BiliBili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

评分 Top10

排名	番剧	评分
1	孤独摇滚!	9.9
2	轻音少女 第一季	9.9
3	凸变英雄X	9.6
4	关于我转生变成史莱姆这档事	9.4

BiliBili番剧库

首页 新增番剧 制作单位 统计 审计日志 登出

制作单位番剧数量

制作单位	番剧数量
8bit	1
BeDream	1
CloverWorks	1
京都アニメーション（京都动画）	1

备注